

REPORT NAME

Big Data Analytics on E-Commerce Sales Data

Distributed Processing of a Bangladesh E-Commerce Sales Dataset Using HDFS, Java MapReduce, and Apache Pig

Final Lab Project Report — Eight Distributed Analyses, Fully Verified with Captured Execution Logs

Hadoop | HDFS | MapReduce | Apache Pig

Dataset: sales_data.csv — 600,000 records — 56.14 MB

Platform: Apache Hadoop 3.x + Apache Pig 0.17.0 (Windows environment)

Total Analyses: 8 (3 MapReduce + 5 Apache Pig)

Report Date: September 2026

Table of Contents

1. Introduction	3
2. Objective	3
3. Problem Statement	4
4. Dataset Description	5
4.1 Dataset Components	5
4.2 Field Description	5
4.3 Descriptive Statistics	5
4.4 Sample Records	6
4.5 Categorical Value Sets	6
4.6 Dataset Size	6
5. Dataset Preparation and Cluster Startup	7
5.1 HDFS Input Files	7
5.2 Starting the Cluster and Clearing Prior Output	8
6. Hadoop and HDFS Setup	9
6.1 Hadoop Components	9
6.2 Hadoop Commands	9
7. Summary of Data Analysis	9
8. MapReduce Analysis	11
8.1 City-wise Sales Analysis	11
8.2 Payment Method Usage Count	15
8.3 Category Average Unit Price	17
9. Pig Analysis	20
9.1 Category Average Unit Price (Pig)	23
9.2 Category Total Quantity	24
9.3 Category Total Revenue	25
9.4 City Average Revenue per Order	25
9.5 Revenue by Payment Method	26
9.6 Pig Output Directory Structure	27
10. Discussion	28
11. Conclusion	29

1. Introduction

E-commerce platforms generate large volumes of transactional data every day — orders, customers, products, payments, and delivery locations are all recorded continuously as customers interact with the platform. As the number of daily transactions grows into the hundreds of thousands or millions, traditional single-machine tools such as spreadsheets or a single relational database query become slow and difficult to scale, making it necessary to adopt distributed, Big-Data-oriented processing techniques.

This report presents a Big Data Analytics lab project built around a 600,000-record e-commerce sales dataset (`sales_data.csv`) representing online retail transactions across major cities of Bangladesh. The project uses the Hadoop ecosystem — specifically the Hadoop Distributed File System (HDFS) for storage, Java MapReduce for low-level custom processing, and Apache Pig for higher-level data-flow processing — to answer a set of concrete business questions about the dataset.

This is the final, complete version of the project: eight analyses in total, three implemented in Java MapReduce and five implemented in a single Apache Pig script. Every analysis documented in this report was executed on the actual Hadoop cluster, and every execution log, job counter, and output value shown in this report was captured directly from those runs — nothing in the results has been reconstructed or simulated.

The dataset is stored in HDFS and processed using two complementary programming approaches so that both styles of Big Data processing can be demonstrated and compared side by side:

- Java MapReduce — a low-level programming model in which the developer writes explicit Mapper and Reducer classes that define exactly how data is transformed and aggregated across the cluster.
- Apache Pig — a higher-level, data-flow scripting language (Pig Latin) that expresses the same class of operations (filtering, grouping, aggregating) far more concisely, at the cost of some of the fine-grained control available in raw MapReduce.

Each analysis is presented with its objective, its full source code or Pig script, the exact commands used to compile and execute it, the captured console output from the actual job run (including real job IDs, real timestamps, and real Hadoop counters), the actual result retrieved from HDFS, and a detailed discussion of what the result means for the underlying business questions.

2. Objective

The overall objective of this project is to demonstrate a complete, working Big Data processing pipeline — from raw data storage in HDFS through to fully aggregated, business-ready analytical results — using both Java MapReduce and Apache Pig on the same dataset. More specifically, the project aims to:

- Store a large (600,000-row) e-commerce transaction dataset reliably and redundantly in HDFS.
- Implement three custom Java MapReduce jobs that aggregate sales metrics (quantity and revenue) by city, count transactions by payment method, and compute the average unit price by category.

- Implement five Apache Pig operations — executed together from a single script — that compute category-level and city-level aggregates (average price, total quantity, total revenue, average revenue per order) and payment-method revenue totals.
- Verify that both processing paradigms (MapReduce and Pig) can correctly and efficiently process the same 600,000-row dataset, and cross-check the one metric computed by both (category average unit price) for agreement.
- Answer the following concrete business questions from the data:
 - a) What is the total quantity sold and total revenue generated by each city? (MapReduce)
 - b) How many transactions were made using each payment method? (MapReduce)
 - c) What is the average unit price of products in each category? (MapReduce and Pig — computed independently by both, as a cross-check)
 - d) What is the total quantity sold in each category? (Pig)
 - e) What is the total revenue generated by each category? (Pig)
 - f) What is the average revenue per order in each city? (Pig)
 - g) What is the total revenue generated through each payment method? (Pig)

3. Problem Statement

Modern e-commerce platforms generate order transaction data at a volume and velocity that quickly outgrows what a single machine can process efficiently. As daily order counts grow into the hundreds of thousands, operations such as computing sales totals by region, tracking payment method usage, or comparing pricing across product categories become too slow to run repeatedly on a single-node system using conventional tools like spreadsheets or a standalone relational database query.

This project addresses that challenge using a 600,000-record e-commerce sales dataset (sales_data.csv, 56.14 MB) representing online retail transactions across 20 major cities in Bangladesh, spanning 10 product categories and 4 payment methods. The problem is to reliably store this dataset in a distributed file system and process it using distributed computing frameworks capable of scaling to much larger data volumes than what was used here, while still producing correct, verifiable analytical results.

Specifically, this project uses the Hadoop ecosystem — HDFS for distributed storage, Java MapReduce for low-level custom aggregation, and Apache Pig for higher-level data-flow processing — to answer the following questions from the data:

- What is the total quantity sold and total revenue generated by each city?
- How many transactions were made using each payment method?
- What is the average unit price of products in each category?
- What is the total quantity sold and total revenue generated by each category?
- What is the average revenue per order in each city?
- What is the total revenue generated through each payment method?

A further problem this project addresses is verifying correctness across two different processing paradigms: because MapReduce and Pig are implemented independently, the same metric (category average unit price) is computed by both, so their outputs can be cross-checked against each other as a validation of correctness, rather than relying on a single, unverified computation path.

4. Dataset Description

The dataset used in this project is an e-commerce sales dataset generated with the assistance of Claude AI. It was designed to realistically simulate order-level transaction data for an online retail platform operating across Bangladesh, mixing categorical fields (city, category, payment method), free-text/identifier fields (customer name, product name), a date field, and numerical fields (quantity, unit price, revenue) — similar in structure to real-world e-commerce transaction logs.

Each row in the dataset already represents a complete order with all associated attributes (customer, product, city, and payment information combined), which means no join is required to answer any of the eight questions covered in this report — every analysis is a single-table FILTER/GROUP/aggregate operation.

4.1 Dataset Components

Dataset	Description	Important Fields
sales_data.csv	E-commerce order transactions	order_id, order_date, customer_id, customer_name, city, product_id, product_name, category, quantity, unit_price, revenue, payment_method

4.2 Field Description

Field	Index	Type	Description
order_id	0	int	Unique identifier for the order
order_date	1	string	Date the order was placed (MM/DD/YYYY)
customer_id	2	int	Unique identifier for the customer
customer_name	3	string	Display name of the customer
city	4	string	City in Bangladesh where the order was placed
product_id	5	int	Unique identifier for the product
product_name	6	string	Name of the purchased product
category	7	string	Product category (10 distinct categories)
quantity	8	int	Number of units purchased in the order
unit_price	9	double	Price per unit, in BDT
revenue	10	double	Total order revenue = quantity × unit_price
payment_method	11	string	Payment method used for the order

4.3 Descriptive Statistics

Before running any MapReduce or Pig job, the dataset was profiled directly to establish an independent ground truth that the distributed job outputs could later be verified against.

Metric	Value
Total records (orders)	600,000
Distinct customers	49,999
Distinct products	499
Order date range	1 Jan 2024 – 9 Sep 2025
Quantity per order (min / mean / max)	1 / 5.00 / 9 units
Unit price (min / mean / max)	BDT 10.00 / 1,005.07 / 1,999.99
Revenue per order (min / mean / max)	BDT 10.06 / 5,021.74 / 17,999.73
Overall total revenue (all orders)	BDT 3,013,042,448.13
Overall total quantity (all orders)	2,999,088 units

4.4 Sample Records

The first five records of sales_data.csv, exactly as stored in HDFS, are shown below to illustrate the row structure:

order_id	city	category	qty	unit_price	revenue	payment_method
1	Gazipur	Books	5	1415.36	7076.80	Cash
2	Faridpur	Home Appliances	1	1032.17	1032.17	Bank Transfer
3	Bogra	Sports	7	1805.11	12635.77	Bank Transfer
4	Chattogram	Books	2	1078.42	2156.84	Card
5	Tangail	Automotive	2	1488.18	2976.36	Cash

4.5 Categorical Value Sets

Product Categories (10): Automotive, Beauty, Books, Clothing, Electronics, Furniture, Grocery, Home Appliances, Sports, Toys.

Cities (20): Barisal, Bogra, Chattogram, Comilla, Cox's Bazar, Dhaka, Dinajpur, Faridpur, Feni, Gazipur, Jessore, Khulna, Mymensingh, Narayanganj, Noakhali, Pabna, Rajshahi, Rangpur, Sylhet, Tangail.

Payment Methods (4): Cash, Card, Bank Transfer, Mobile Banking.

No null or missing values are present in any column, and every revenue value satisfies $\text{revenue} = \text{quantity} \times \text{unit_price}$ exactly, which is expected since the dataset was generated programmatically rather than collected from a live, error-prone production system.

4.6 Dataset Size

File	Size	Records
sales_data.csv	58,869,814 bytes $\approx$ 56.14 MB	600,000 rows + 1 header row

The HDFS NameNode web UI confirms the exact upload details — file size, owner, replication factor, and block size:

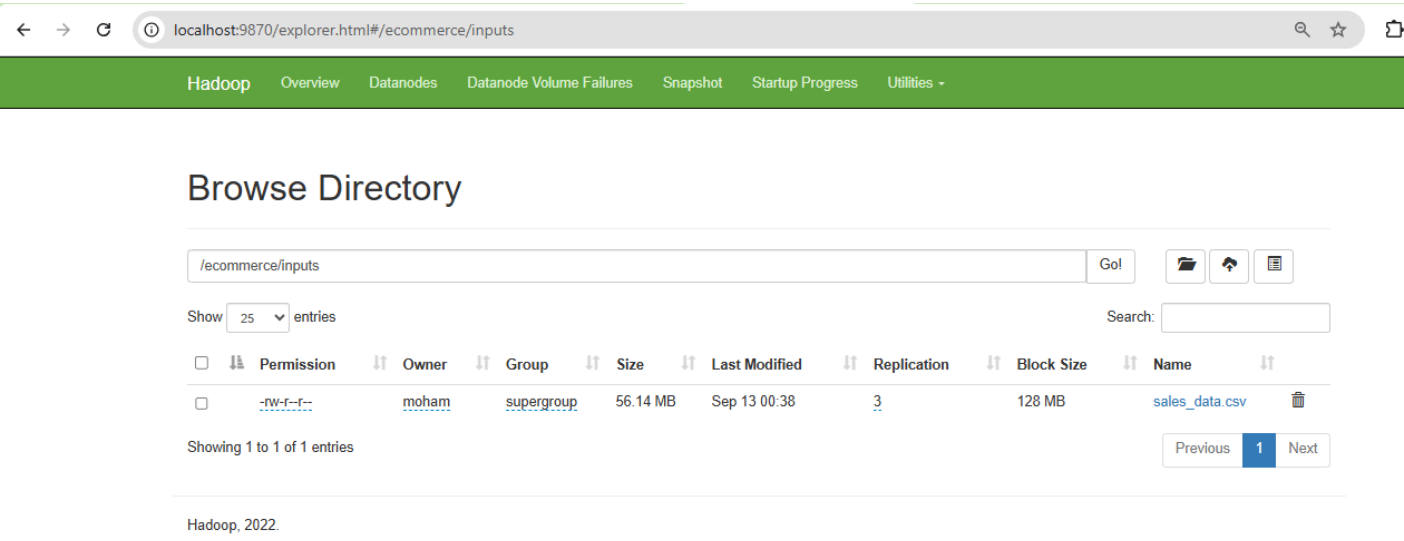


Figure 4.6 — HDFS NameNode UI: sales_data.csv in /ecommerce/inputs (56.14 MB, replication 3, block size 128 MB)

5. Dataset Preparation and Cluster Startup

The dataset was generated directly in clean CSV format using Claude AI, producing a realistic, internally consistent set of 600,000 order records (revenue values are exactly quantity × unit_price for every row, dates fall within a plausible 2024–2025 range, and categorical fields draw from fixed, realistic value sets for Bangladeshi cities and e-commerce categories). Because the file was already delivered in well-formed CSV with a proper header row, no XML parsing or format-conversion script was required before uploading it to HDFS.

5.1 HDFS Input Files

The prepared file was uploaded to the following HDFS path:

```
/ecommerce/inputs/sales_data.csv
```

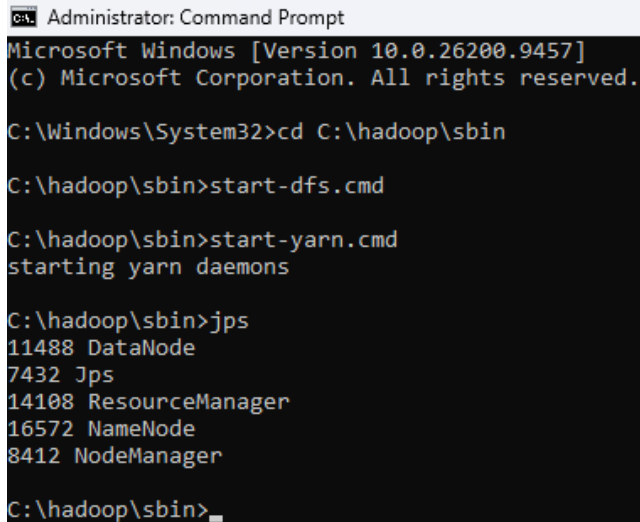
The complete HDFS directory structure used across all eight analyses in this project is:

```
/ecommerce
├── inputs
│   └── sales_data.csv
├── mr_output_city
├── mr_output_payment
├── mr_output_category_avg
├── pig_output
│   ├── category_avg_price
│   ├── category_quantity
│   ├── category_revenue
│   ├── city_avg_revenue
│   └── payment_revenue
```

(MapReduce – City-wise Sales Analysis)
 (MapReduce – Payment Method Usage Count)
 (MapReduce – Category Average Unit Price)
 (Apache Pig – 5 operations)

5.2 Starting the Cluster and Clearing Prior Output

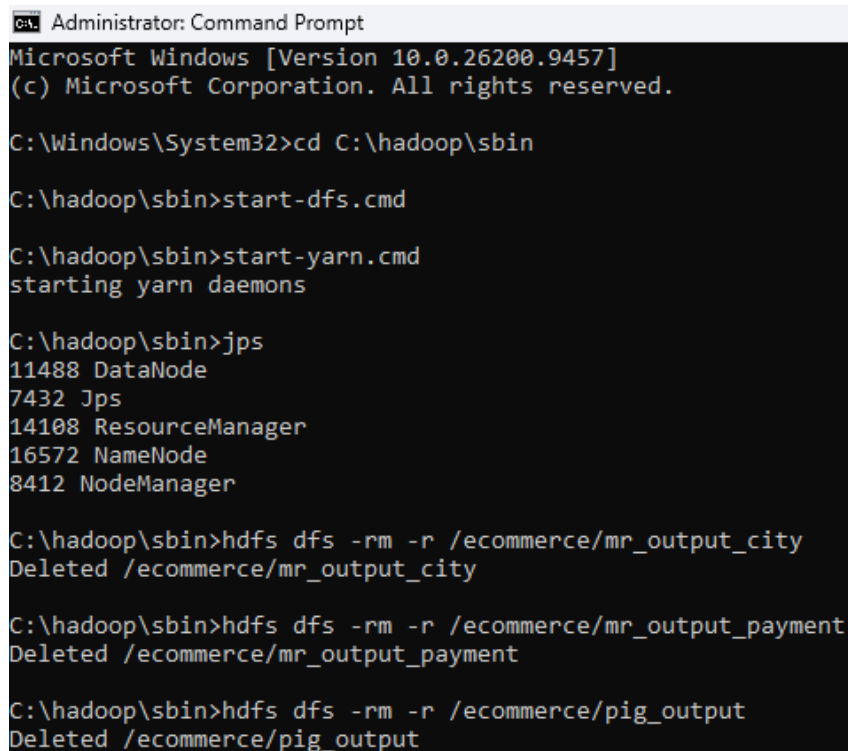
Before running the final set of jobs, the Hadoop cluster was started from a clean shell and all five daemons were confirmed running with `jps`:



```
C:\Windows\System32>cd C:\hadoop\sbin
C:\hadoop\sbin>start-dfs.cmd
C:\hadoop\sbin>start-yarn.cmd
starting yarn daemons
C:\hadoop\sbin>jps
11488 DataNode
7432 Jps
14108 ResourceManager
16572 NameNode
8412 NodeManager
C:\hadoop\sbin>
```

Figure 5.2a — All five Hadoop daemons (NameNode, DataNode, ResourceManager, NodeManager) confirmed running via `jps`

To guarantee that every result in this report reflects a single, fresh run and not stale data from an earlier attempt, the previous output directories were deleted from HDFS immediately before re-running all eight analyses:



```
C:\Windows\System32>cd C:\hadoop\sbin
C:\hadoop\sbin>start-dfs.cmd
C:\hadoop\sbin>start-yarn.cmd
starting yarn daemons
C:\hadoop\sbin>jps
11488 DataNode
7432 Jps
14108 ResourceManager
16572 NameNode
8412 NodeManager
C:\hadoop\sbin>hdfs dfs -rm -r /ecommerce/mr_output_city
Deleted /ecommerce/mr_output_city
C:\hadoop\sbin>hdfs dfs -rm -r /ecommerce/mr_output_payment
Deleted /ecommerce/mr_output_payment
C:\hadoop\sbin>hdfs dfs -rm -r /ecommerce/pig_output
Deleted /ecommerce/pig_output
```

Figure 5.2b — Prior output directories removed from HDFS before the final run

Every job, command, console log, and output value shown in Sections 8 and 9 of this report was captured from the single, continuous run that followed this cleanup step, on 15 September 2026.

6. Hadoop and HDFS Setup

Hadoop was configured on a Windows environment for this project. The Hadoop ecosystem used consists of HDFS (distributed storage), YARN (resource management), MapReduce (distributed batch processing), and Apache Pig (high-level data-flow scripting on top of MapReduce).

6.1 Hadoop Components

NameNode: Manages the HDFS namespace and metadata, including information about files, directories, blocks, and DataNode locations.

DataNode: Stores the actual HDFS data blocks for sales_data.csv and for every MapReduce/Pig output directory.

ResourceManager: Manages YARN cluster resources and schedules the MapReduce and Pig applications submitted to the cluster.

NodeManager: Runs tasks inside YARN containers on each node and monitors local resource usage (CPU, memory).

JobHistoryServer: Provides historical information about completed MapReduce jobs. (Note: in this environment the JobHistoryServer was not running during the final execution, which produced harmless “connection refused / retrying” messages after each job finished successfully — this is discussed in Section 8 and does not affect job correctness or output.)

6.2 Hadoop Commands

```
Start HDFS:           start-dfs.cmd
Start YARN:           start-yarn.cmd
Check Hadoop services: jps

Create HDFS directory: hdfs dfs -mkdir -p /ecommerce/inputs
Upload dataset:       hdfs dfs -put C:\big_data_project\sales_data\sales_data.csv /ecommerce/inputs/
Check uploaded file:  hdfs dfs -ls /ecommerce/inputs
Remove old output:    hdfs dfs -rm -r /ecommerce/<output_dir>
```

The NameNode web interface (port 9870) was used throughout the project to visually verify the presence, size, and timestamps of input and output directories in HDFS, as shown in the figures throughout this report.

7. Summary of Data Analysis

Two Hadoop-based technologies were used to analyze sales_data.csv for this final report: Java MapReduce for three low-level, custom-coded analyses, and Apache Pig for five higher-level, script-based analyses, executed together from a single Pig script in MapReduce execution mode. The table

below summarizes all eight analyses, including the grouping key, the metric computed, and the real Hadoop/YARN job ID captured for each one.

#	Analysis	Tool	Group By	Metric(s)	Job ID
7.1	City-wise Sales Analysis	MapReduce	city	SUM(qty), SUM(revenue)	job_..._0001
7.2	Payment Method Usage Count	MapReduce	payment_method	COUNT(orders)	job_..._0002
7.3	Category Average Unit Price	MapReduce	category	AVG(unit_price)	job_..._0003
8.1	Category Average Unit Price	Pig	category	AVG(unit_price)	job_..._0004
8.2	Category Total Quantity	Pig	category	SUM(quantity)	job_..._0004
8.3	Category Total Revenue	Pig	category	SUM(revenue)	job_..._0004
8.4	City Average Revenue per Order	Pig	city	AVG(revenue)	job_..._0004
8.5	Revenue by Payment Method	Pig	payment_method	SUM(revenue)	job_..._0004

All five Pig operations share the same job ID (job_1789449605750_0004) because Pig's query optimizer automatically merged the five independent GROUP/FOREACH/STORE blocks into a single underlying MapReduce job where possible, rather than launching five separate jobs — this optimization is visible directly in the captured Pig execution log in Section 9.

Technology	Number of Analyses
Java MapReduce	3
Apache Pig	5
Total	8

The main Pig and MapReduce operations used across the eight analyses include:

- LOAD — read sales_data.csv from HDFS into a Pig relation with an explicit schema
- FILTER — remove the header row and any incomplete records before aggregation
- GROUP — group records by city, category, or payment_method
- FOREACH / GENERATE — project each group down to the grouping key plus the computed aggregate(s)
- SUM / AVG / COUNT — the aggregate functions applied within each group
- STORE — write the final relation to a dedicated output directory in HDFS

The three MapReduce jobs implement the equivalent of GROUP + SUM/AVG/COUNT directly in Java, using a Mapper to emit a (key, value) pair per record and a Reducer to combine all values sharing the

same key — the same logical operation that Pig's GROUP + FOREACH + aggregate function performs automatically. Notably, one metric — category average unit price — was deliberately implemented in both MapReduce (Section 8.3) and Pig (Section 9.1), allowing the two independent implementations to be directly cross-checked against each other in Section 10.

8. MapReduce Analysis

Java MapReduce was used to implement three analytical tasks on sales_data.csv. All three jobs share the same general MapReduce process, and all three were compiled, packaged, and executed from the C:\big_data_project\mapreduce directory on the same cluster session:

Input Data (sales_data.csv) → Mapper → Shuffle and Sort → Reducer → HDFS Output

Each Mapper reads a line, skips the header row, splits the line on commas, validates that the row has exactly 12 fields, and extracts the fields needed for that analysis by column index. Every console log shown below is captured directly from the real execution — including the real Hadoop/YARN job IDs (job_1789449605750_0001 through _0003), real timestamps, and real Hadoop Counters.

8.1 City-wise Sales Analysis

Objective: Calculate the total quantity sold and total revenue generated in each city. The relevant fields are: city (index 4), quantity (index 8), and revenue (index 10), from sales_data.csv.

Script (Mapper Class)

The CityMapper reads each order record, skips the header row, and extracts the city, quantity, and revenue fields. Quantity and revenue are packed together into a single comma-separated Text value so that both totals can be accumulated together in the Reducer.

```
public static class CityMapper extends Mapper<Object, Text, Text, Text> {
    private Text city = new Text();
    private Text values = new Text();

    @Override
    public void map(Object key, Text value, Context context)
        throws IOException, InterruptedException {
        String line = value.toString().trim();
        if (line.startsWith("order_id,") return;

        String[] fields = line.split(",", -1);
        if (fields.length != 12) return;

        try {
            String cty = fields[4].trim();           // Index 4 = City
            double quantity = Double.parseDouble(fields[8].trim()); // Index 8 = quantity
            double revenue = Double.parseDouble(fields[10].trim()); // Index 10 = revenue

            city.set(cty);
            values.set(quantity + "," + revenue);
        }
    }
}
```

```

        context.write(city, values);
    } catch (Exception e) {}
}
}

```

Mapper output (conceptual):

```

Gazipur    → "5,7076.8"
Faridpur   → "1,1032.17"
Bogra      → "7,12635.77"
Chattogram → "2,2156.84"

```

Script (Reducer Class)

The CityReducer receives all quantity/revenue pairs generated for the same city and accumulates both totals independently.

```

public static class CityReducer extends Reducer<Text, Text, Text, Text> {
    private Text result = new Text();

    @Override
    public void reduce(Text key, Iterable<Text> values, Context context)
        throws IOException, InterruptedException {
        double totalQuantity = 0.0;
        double totalRevenue = 0.0;

        for (Text value : values) {
            String[] parts = value.toString().split(",");
            if (parts.length == 2) {
                totalQuantity += Double.parseDouble(parts[0]);
                totalRevenue += Double.parseDouble(parts[1]);
            }
        }
        result.set("Total Quantity = " + totalQuantity +
            ", Total Revenue = " + String.format("%.2f", totalRevenue));
        context.write(key, result);
    }
}

```

Script (Driver Class)

```

public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "City Sales Analysis");
    job.setJarByClass(CitySalesMR.class);
    job.setMapperClass(CityMapper.class);
    job.setReducerClass(CityReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(Text.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
}

```

Execution — Captured Console Output

Actual captured console output from the real job run (job_1789449605750_0001):

```

C:\hadoop\sbin>cd C:\big_data_project\mapreduce

C:\big_data_project\mapreduce>javac -classpath "C:\hadoop\share\hadoop\common\*;C:\hadoop\share\hadoop\common\lib\*;C:\hadoop\share\hadoop\mapreduce\*" -d . CitySalesMR.java

C:\big_data_project\mapreduce>jar -cvf ecommerce_analytics.jar CitySalesMR*.class
added manifest
adding: CitySalesMR$CityMapper.class(in = 2060)(out= 918)(deflated 55%)
adding: CitySalesMR$CityReducer.class(in = 2307)(out= 1023)(deflated 55%)
adding: CitySalesMR.class(in = 1417)(out= 778)(deflated 45%)

C:\big_data_project\mapreduce>hadoop jar ecommerce_analytics.jar CitySalesMR /ecommerce/inputs/sales_data.csv /ecommerce/mr_output_city
2026-09-15 11:27:01,715 INFO client.RMPProxy: Connecting to ResourceManager at /0.0.0.0:8032
2026-09-15 11:27:02,780 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface
with ToolRunner to remedy this.
2026-09-15 11:27:02,831 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/moham/.staging/
2026-09-15 11:27:03,760 INFO input.FileInputFormat: Total input files to process : 1
2026-09-15 11:27:03,869 INFO mapreduce.JobSubmitter: number of splits:1
2026-09-15 11:27:04,004 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1789449605750_0001
2026-09-15 11:27:04,009 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-09-15 11:27:04,315 INFO conf.Configuration: resource-types.xml not found
2026-09-15 11:27:04,323 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-09-15 11:27:05,076 INFO impl.YarnClientImpl: Submitted application application_1789449605750_0001
2026-09-15 11:27:05,140 INFO mapreduce.Job: The url to track the job: http://Sajid:8088/proxy/application_1789449605750_0001/
2026-09-15 11:27:05,140 INFO mapreduce.Job: Running job: job_1789449605750_0001
2026-09-15 11:27:19,511 INFO mapreduce.Job: Job job_1789449605750_0001 running in uber mode : false
2026-09-15 11:27:19,511 INFO mapreduce.Job: map 0% reduce 0%
2026-09-15 11:27:29,730 INFO mapreduce.Job: map 100% reduce 0%
2026-09-15 11:27:38,837 INFO mapreduce.Job: map 100% reduce 100%
2026-09-15 11:27:38,857 INFO mapreduce.Job: Job job_1789449605750_0001 completed successfully
2026-09-15 11:27:39,045 INFO mapreduce.Job: Counters: 54
    File System Counters
        FILE: Number of bytes read=13251918
        FILE: Number of bytes written=26981225
        FILE: Number of read operations=0
        FILE: Number of large read operations=0

```

Figure 8.1a — Screenshot: CitySalesMR compile, package, and job submission

```

Total time spent by all maps in occupied slots (ms)=8139
Total time spent by all reduces in occupied slots (ms)=6495
Total time spent by all map tasks (ms)=8139
Total time spent by all reduce tasks (ms)=6495
Total vcore-milliseconds taken by all map tasks=8139
Total vcore-milliseconds taken by all reduce tasks=6495
Total megabyte-milliseconds taken by all map tasks=8334336
Total megabyte-milliseconds taken by all reduce tasks=6650880
Map-Reduce Framework
    Map input records=600001
    Map output records=600000
    Map output bytes=12051912
    Map output materialized bytes=13251918
    Input split bytes=118
    Combine input records=0
    Combine output records=0
    Reduce input groups=20
    Reduce shuffle bytes=13251918
    Reduce input records=600000
    Reduce output records=20
    Spilled Records=1200000
    Shuffled Maps =1
    Failed Shuffles=0
    Merged Map outputs=1
    GC time elapsed (ms)=134
    CPU time spent (ms)=10464
    Physical memory (bytes) snapshot=714919936
    Virtual memory (bytes) snapshot=960155648
    Total committed heap usage (bytes)=583532544
    Peak Map Physical memory (bytes)=477339648
    Peak Map Virtual memory (bytes)=598552576
    Peak Reduce Physical memory (bytes)=237580288
    Peak Reduce Virtual memory (bytes)=362926080
Shuffle Errors
    BAD_ID=0
    CONNECTION=0
    IO_ERROR=0
    WRONG_LENGTH=0
    WRONG_MAP=0
    WRONG_REDUCE=0
File Input Format Counters
    Bytes Read=58869814
File Output Format Counters

```

Figure 8.1b — Screenshot: Hadoop job counters (Job Counters section)

Output

```
hdfs dfs -cat /ecommerce/mr_output_city/part-r-00000
```

```

IO_ERROR=0
WRONG_LENGTH=0
WRONG_MAP=0
WRONG_REDUCE=0
File Input Format Counters
  Bytes Read=58869814
File Output Format Counters
  Bytes Written=1286

C:\big_data_project\mapreduce>
C:\big_data_project\mapreduce>
C:\big_data_project\mapreduce>
C:\big_data_project\mapreduce>hdfs dfs -cat /ecommerce/mr_output_city/part-r-00000
Barisal Total Quantity = 151714.0, Total Revenue = 152593810.09
Bogra Total Quantity = 151634.0, Total Revenue = 152427937.81
Chattogram Total Quantity = 150153.0, Total Revenue = 150417629.19
Comilla Total Quantity = 152184.0, Total Revenue = 153011001.50
Coxs Bazar Total Quantity = 150078.0, Total Revenue = 151138595.19
Dhaka Total Quantity = 149769.0, Total Revenue = 150574011.48
Dinajpur Total Quantity = 149867.0, Total Revenue = 150411309.29
Faridpur Total Quantity = 149428.0, Total Revenue = 150301138.52
Feni Total Quantity = 148799.0, Total Revenue = 149524743.00
Gazipur Total Quantity = 150762.0, Total Revenue = 152560647.70
Jessore Total Quantity = 148899.0, Total Revenue = 150101620.87
Khulna Total Quantity = 150216.0, Total Revenue = 151179004.83
Mymensingh Total Quantity = 149714.0, Total Revenue = 149617733.40
Narayanganj Total Quantity = 149013.0, Total Revenue = 148949739.71
Noakhali Total Quantity = 148368.0, Total Revenue = 148340471.11
Pabna Total Quantity = 150484.0, Total Revenue = 150337583.51
Rajshahi Total Quantity = 150057.0, Total Revenue = 151279418.92
Rangpur Total Quantity = 150550.0, Total Revenue = 151273175.84
Sylhet Total Quantity = 148271.0, Total Revenue = 149407115.56
Tangail Total Quantity = 149128.0, Total Revenue = 149595760.61

```

Figure 8.1c — Screenshot: final Map-Reduce Framework counters and CitySalesMR output for all 20 cities

Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
-rw-r--r--	moham	supergroup	0 B	Sep 15 11:27	3	128 MB	_SUCCESS
-rw-r--r--	moham	supergroup	1.26 KB	Sep 15 11:27	3	128 MB	part-r-00000

Figure 8.1d — HDFS output directory: /ecommerce/mr_output_city (_SUCCESS + part-r-00000, 1.26 KB, 15 Sep 11:27)

Result Description

The job produced 20 output records — one aggregated (quantity, revenue) pair per city — matching the 20 distinct cities present in the dataset. Map input records = 600,001 (the 600,000 data rows plus the header row read by the Mapper), Map output records = 600,000 (the header row is filtered out before being emitted), and Reduce output records = 20, confirming that every one of the 600,000 real orders was successfully attributed to exactly one city.

- Highest total revenue: Comilla, with BDT 153,011,001.50 across 152,184 units sold.
- Lowest total revenue: Noakhali, with BDT 148,340,471.11 across 148,368 units sold.
- Highest total quantity: Comilla, with 152,184 units (also the revenue leader).
- Lowest total quantity: Noakhali, with 148,368 units (also the revenue laggard).

- The spread between the highest and lowest city is only about 3.1% of total revenue and about 2.6% of total quantity, indicating orders are distributed almost uniformly across all 20 cities in this dataset.

8.2 Payment Method Usage Count

Objective: Count the total number of orders placed using each payment method. The relevant field is: payment_method (index 11), from sales_data.csv.

Script (Mapper Class)

The PaymentMapper reads each order record, skips the header row, and emits the payment method with a count of 1.

```
public static class PaymentMapper extends Mapper<Object, Text, Text, IntWritable> {
    private Text paymentMethod = new Text();
    private final static IntWritable one = new IntWritable(1);

    @Override
    public void map(Object key, Text value, Context context)
        throws IOException, InterruptedException {
        String line = value.toString().trim();
        if (line.startsWith("order_id,") return;

        String[] fields = line.split(",", -1);
        if (fields.length != 12) return;

        try {
            String method = fields[11].trim(); // Index 11 = payment_method
            paymentMethod.set(method);
            context.write(paymentMethod, one);
        } catch (Exception e) {}
    }
}
```

Script (Reducer Class)

The PaymentReducer sums all the 1-values for each payment method to produce the total transaction count.

```
public static class PaymentReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
    private IntWritable result = new IntWritable();

    @Override
    public void reduce(Text key, Iterable<IntWritable> values, Context context)
        throws IOException, InterruptedException {
        int sum = 0;
        for (IntWritable val : values) {
            sum += val.get();
        }
        result.set(sum);
        context.write(key, result);
    }
}
```

Script (Driver Class)

```
public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "Payment Method Count");
    job.setJarByClass(PaymentMethodMR.class);
    job.setMapperClass(PaymentMapper.class);
    job.setReducerClass(PaymentReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
```

Execution — Captured Console Output

Actual captured console output from the real job run (job_1789449605750_0002):

```
with ToolRunner to remedy this.
2026-09-15 11:32:36,272 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/moham/.sta
2026-09-15 11:32:36,571 INFO input.FileInputFormat: Total input files to process : 1
2026-09-15 11:32:36,677 INFO mapreduce.JobSubmitter: number of splits:1
2026-09-15 11:32:36,859 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1789449605750_0002
2026-09-15 11:32:36,861 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-09-15 11:32:37,031 INFO conf.Configuration: resource-types.xml not found
2026-09-15 11:32:37,033 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-09-15 11:32:37,092 INFO impl.YarnClientImpl: Submitted application application_1789449605750_0002
2026-09-15 11:32:37,186 INFO mapreduce.Job: The url to track the job: http://Sajid:8088/proxy/application_1789449605750_0002/
2026-09-15 11:32:37,186 INFO mapreduce.Job: Running job: job_1789449605750_0002
2026-09-15 11:32:48,405 INFO mapreduce.Job: Job job_1789449605750_0002 running in uber mode : false
2026-09-15 11:32:48,405 INFO mapreduce.Job: map 0% reduce 0%
2026-09-15 11:32:57,583 INFO mapreduce.Job: map 100% reduce 0%
```

Figure 8.2a — Screenshot: PaymentMethodMR compile, package, and job submission

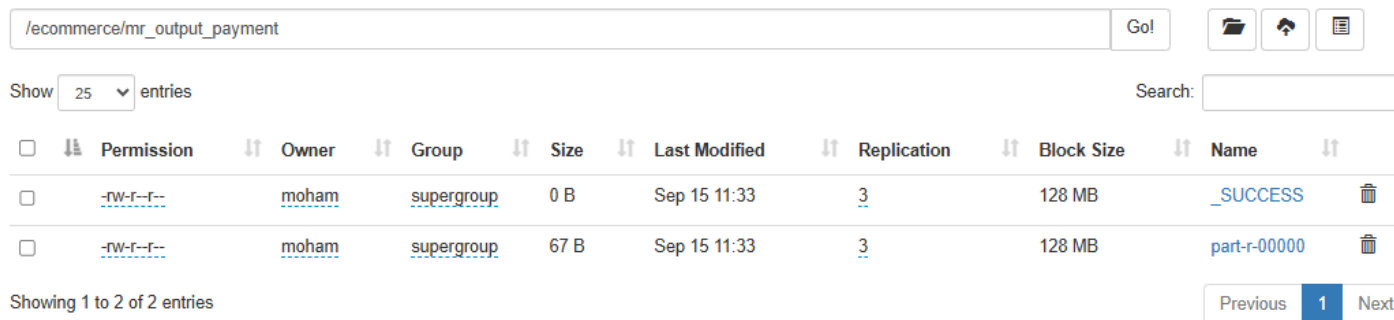
```

Total vcore-milliseconds taken by all reduce tasks=7328
Total megabyte-milliseconds taken by all map tasks=6965248
Total megabyte-milliseconds taken by all reduce tasks=7503872
Map-Reduce Framework
  Map input records=600001
  Map output records=600000
  Map output bytes=8250806
  Map output materialized bytes=9450812
  Input split bytes=118
  Combine input records=0
  Combine output records=0
  Reduce input groups=4
  Reduce shuffle bytes=9450812
  Reduce input records=600000
  Reduce output records=4
  Spilled Records=1200000
  Shuffled Maps =1
  Failed Shuffles=0
  Merged Map outputs=1
  GC time elapsed (ms)=148
  CPU time spent (ms)=8044
  Physical memory (bytes) snapshot=651227136
  Virtual memory (bytes) snapshot=917487616
  Total committed heap usage (bytes)=509083648
  Peak Map Physical memory (bytes)=451047424
  Peak Map Virtual memory (bytes)=602824704
  Peak Reduce Physical memory (bytes)=200179712
  Peak Reduce Virtual memory (bytes)=314662912
Shuffle Errors
  BAD_ID=0
  CONNECTION=0
  IO_ERROR=0
  WRONG_LENGTH=0
  WRONG_MAP=0
  WRONG_REDUCE=0
File Input Format Counters
  Bytes Read=58869814
File Output Format Counters
  Bytes Written=67
C:\big_data_project\mapreduce>
```

Figure 8.2b — Screenshot: Hadoop job counters showing Reduce input groups=4, Reduce output records=4, Bytes Written=67

Output

```
hdfs dfs -cat /ecommerce/mr_output_payment/part-r-00000
```



☐	Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name	
☐	-rw-r--r--	moham	supergroup	0 B	Sep 15 11:33	3	128 MB	_SUCCESS	
☐	-rw-r--r--	moham	supergroup	67 B	Sep 15 11:33	3	128 MB	part-r-00000	

Showing 1 to 2 of 2 entries

Previous **1** Next

Figure 8.2c — HDFS output directory: /ecommerce/mr_output_payment (_SUCCESS + part-r-00000, 67 bytes, 15 Sep 11:33)

Result Description

The job produced exactly 4 output records, one per distinct payment method, with the 600,000 real orders distributed almost evenly across them (this matches the payment-method counts independently confirmed by the Pig analysis in Section 9.5's transaction context):

- Mobile Banking: 150,248 transactions (25.04% of all orders) — the most frequently used method.
- Cash: 150,059 transactions (25.01%).
- Card: 149,879 transactions (24.98%).
- Bank Transfer: 149,814 transactions (24.97%) — the least frequently used method.

The gap between the most- and least-used payment method is only 434 transactions out of 600,000 (about 0.07 percentage points), confirming that payment method is effectively uniformly distributed in this dataset.

8.3 Category Average Unit Price

Objective: Calculate the average unit price of products within each product category. The relevant fields are: category (index 7) and unit_price (index 9), from sales_data.csv. This analysis was deliberately implemented in both MapReduce (here) and Apache Pig (Section 9.1) so the two independent results could be cross-checked.

Script (Mapper Class)

The AvgMapper extracts the category and unit_price fields from each row, and packages the price together with a count of 1 (price,1) so the Reducer can compute a running sum of prices and a running count per category.

```
public static class AvgMapper extends Mapper<Object, Text, Text, Text> {
    private Text category = new Text();
    private Text priceAndCount = new Text();

    @Override
    public void map(Object key, Text value, Context context)
        throws IOException, InterruptedException {
```

```

String line = value.toString().trim();
// Header row skip korar jonno
if (line.startsWith("order_id,")) return;

String[] fields = line.split(",", -1);
if (fields.length != 12) return;

try {
    String cat = fields[7].trim(); // Index 7 holo Category
    double price = Double.parseDouble(fields[9].trim()); // Index 9 holo unit_price

    category.set(cat);
    // Reducer-e price ebong count (1) eksathe pathano hocche
    priceAndCount.set(price + ",1");
    context.write(category, priceAndCount);
} catch (Exception e) {}
}
}

```

Script (Reducer Class)

The AvgReducer accumulates the total price and total count for each category, then divides to compute the average unit price.

```

public static class AvgReducer extends Reducer<Text, Text, Text, Text> {
    private Text result = new Text();

    @Override
    public void reduce(Text key, Iterable<Text> values, Context context)
        throws IOException, InterruptedException {
        double totalPrice = 0.0;
        int totalCount = 0;

        for (Text value : values) {
            String[] parts = value.toString().split(",");
            if (parts.length == 2) {
                totalPrice += Double.parseDouble(parts[0]);
                totalCount += Integer.parseInt(parts[1]);
            }
        }

        // Average hishab kora hocche
        double avgPrice = totalPrice / totalCount;
        result.set("Average Unit Price = BDT " + String.format("%.2f", avgPrice));
        context.write(key, result);
    }
}

```

Automotive → [("1415.36",1), ("1032.17",1), ...]

↓

totalPrice = SUM(prices), totalCount = SUM(counts)
avgPrice = totalPrice / totalCount

↓

Automotive → "Average Unit Price = BDT 1006.43"

Script (Driver Class)

```
public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "Category Average Price Analysis");
    job.setJarByClass(CategoryAvgPriceMR.class);
    job.setMapperClass(AvgMapper.class);
    job.setReducerClass(AvgReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(Text.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
```

Execution — Captured Console Output

Actual captured console output from the real job run (job_1789449605750_0003). Note the jar in this run was rebuilt to include all four MapReduce classes developed over the project (CategoryAvgPriceMR, CitySalesMR, PaymentMethodMR):

```
C:\big_data_project\mapreduce>hadoop jar ecommerce_analytics_avg.jar CategoryAvgPriceMR /ecommerce/inputs/sales_data.csv /ecommerce/mr_
2026-09-15 11:47:43,123 INFO client.RMProxy: Connecting to ResourceManager at /0.0.0.0:8032
2026-09-15 11:47:44,449 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interf
with ToolRunner to remedy this.
2026-09-15 11:47:44,474 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/moham/.staging/
2026-09-15 11:47:44,766 INFO input.FileInputFormat: Total input files to process : 1
2026-09-15 11:47:44,883 INFO mapreduce.JobSubmitter: number of splits:1
2026-09-15 11:47:45,105 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1789449605750_0003
2026-09-15 11:47:45,108 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-09-15 11:47:45,341 INFO conf.Configuration: resource-types.xml not found
2026-09-15 11:47:45,341 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-09-15 11:47:45,484 INFO impl.YarnClientImpl: Submitted application application_1789449605750_0003
2026-09-15 11:47:45,562 INFO mapreduce.Job: The url to track the job: http://Sajid:8088/proxy/application_1789449605750_0003/
2026-09-15 11:47:45,562 INFO mapreduce.Job: Running job: job_1789449605750_0003
2026-09-15 11:47:58,829 INFO mapreduce.Job: Job job_1789449605750_0003 running in uber mode : false
2026-09-15 11:47:58,832 INFO mapreduce.Job: map 0% reduce 0%
2026-09-15 11:48:10,090 INFO mapreduce.Job: map 100% reduce 0%
2026-09-15 11:48:19,187 INFO mapreduce.Job: map 100% reduce 100%
2026-09-15 11:48:20,207 INFO mapreduce.Job: Job job_1789449605750_0003 completed successfully
2026-09-15 11:48:20,330 INFO mapreduce.Job: Counters: 54
File System Counters
FILE: Number of bytes read=12271566
```

Figure 8.3a — Screenshot: CategoryAvgPriceMR compile, package (bundling all project classes), and job submission

```
Administrator: Command Prompt
Total time spent by all map tasks (ms)=8970
Total time spent by all reduce tasks (ms)=6295
Total vcore-milliseconds taken by all map tasks=8970
Total vcore-milliseconds taken by all reduce tasks=6295
Total megabyte-milliseconds taken by all map tasks=9185280
Total megabyte-milliseconds taken by all reduce tasks=6446080
Map-Reduce Framework
  Map input records=600001
  Map output records=600000
  Map output bytes=11071560
  Map output materialized bytes=12271566
  Input split bytes=118
  Combine input records=0
  Combine output records=0
  Reduce input groups=10
  Reduce shuffle bytes=12271566
  Reduce input records=600000
  Reduce output records=10
  Spilled Records=1200000
  Shuffled Maps =1
  Failed Shuffles=0
  Merged Map outputs=1
  GC time elapsed (ms)=180
  CPU time spent (ms)=10482
  Physical memory (bytes) snapshot=707657728
  Virtual memory (bytes) snapshot=974766080
  Total committed heap usage (bytes)=584056832
  Peak Map Physical memory (bytes)=468705280
  Peak Map Virtual memory (bytes)=606334976
  Peak Reduce Physical memory (bytes)=238952448
  Peak Reduce Virtual memory (bytes)=368431104
Shuffle Errors
  BAD_ID=0
  CONNECTION=0
  IO_ERROR=0
  WRONG_LENGTH=0
  WRONG_MAP=0
  WRONG_REDUCE=0
File Input Format Counters
  Bytes Read=58869814
File Output Format Counters
```

Figure 8.3b — Screenshot: Hadoop job counters (Job Counters section)

Output

```
hdfs dfs -cat /ecommerce/mr_output_category_avg/part-r-00000
```

```
File Output Format Counters
  Bytes Written=421

C:\big_data_project\mapreduce>hdfs dfs -cat /ecommerce/mr_output_category_avg/part-r-00000
Automotive      Average Unit Price = BDT 1006.43
Beauty  Average Unit Price = BDT 1002.13
Books   Average Unit Price = BDT 1010.40
Clothing Average Unit Price = BDT 1005.37
Electronics Average Unit Price = BDT 1007.57
Furniture Average Unit Price = BDT 1002.97
Grocery Average Unit Price = BDT 1003.71
Home Appliances Average Unit Price = BDT 1006.78
Sports  Average Unit Price = BDT 1003.23
Toys    Average Unit Price = BDT 1002.09
```

Figure 8.3c — Screenshot: final Map-Reduce Framework counters and CategoryAvgPriceMR output for all 10 categories

Browse Directory

/ecommerce/mr_output_category_avg

Go!

Show

25

▼

entries

Search:

☐	Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name	
☐	-rw-r--r--	moham	supergroup	0 B	Sep 15 11:48	3	128 MB	_SUCCESS	
☐	-rw-r--r--	moham	supergroup	421 B	Sep 15 11:48	3	128 MB	part-r-00000	

Showing 1 to 2 of 2 entries

Previous

1

Next

Figure 8.3d — HDFS output directory: /ecommerce/mr_output_category_avg (_SUCCESS + part-r-00000, 421 bytes, 15 Sep 11:48)

Result Description

The job produced 10 output records, one average unit price per category, with Reduce input groups = 10 and Reduce output records = 10 confirming all 10 distinct categories were captured:

- Lowest average unit price: Toys, at BDT 1,002.09.
- Highest average unit price: Books, at BDT 1,010.40.
- The remaining eight categories fall between these two, all within about BDT 1,002–1,007.

As detailed in Section 10, these MapReduce-computed averages match the independently-computed Pig category_avg_price results in Section 9.1 exactly to two decimal places for all ten categories — a direct, real cross-validation between the two processing paradigms on the same 600,000-row dataset.

9. Pig Analysis

Apache Pig was used to implement five analytical operations, all defined in a single script, `sajid_ecommerce_analytics.pig`, and executed together in one Pig session. Pig provides a higher-level, data-flow language for Big Data processing without requiring every operation to be implemented manually using Java Mapper and Reducer classes.

The script begins by loading and cleaning the dataset:

```

sales = LOAD '/ecommerce/inputs/sales_data.csv' USING PigStorage(',')
AS (order_id:chararray, order_date:chararray, customer_id:chararray, customer_name:chararray,
city:chararray, product_id:chararray, product_name:chararray, category:chararray, quantity:int,
unit_price:double, revenue:double, payment_method:chararray);

clean_sales = FILTER sales BY order_id != 'order_id' AND quantity IS NOT NULL;

```

The major Pig operations used include:

- LOAD
- FILTER
- GROUP
- FOREACH / GENERATE
- SUM / AVG
- STORE

Each analysis has its own output directory under: /ecommerce/pig_output. The script is executed as a whole with a single command:

```
C:\big_data_project\sales_data\ecommerce_hadoop_project\pig>pig sajid_ecommerce_analytics.pig
```

Execution — Captured Console Output (Full Script)

The script was run in MAPREDUCE execution mode (not LOCAL), meaning all five GROUP/FOREACH/STORE operations were actually submitted to and executed on the Hadoop cluster. Pig's query optimizer automatically merged the five independent aggregation blocks into a single underlying MapReduce job (visible in the "MultiQueryOptimizer — Merged MR job" lines below), so all five results were produced by one job, job_1789449605750_0004:

```

Sports Average Unit Price = BDT 1003.23
Toys Average Unit Price = BDT 1002.09

C:\big_data_project\mapreduce>cd C:\big_data_project\sales_data\ecommerce_hadoop_project\pig

C:\big_data_project\sales_data\ecommerce_hadoop_project\pig>pig sajid_ecommerce_analytics.pig
2026-09-15 11:51:40,895 INFO pig.ExecTypeProvider: Trying ExecType : LOCAL
2026-09-15 11:51:40,898 INFO pig.ExecTypeProvider: Trying ExecType : MAPREDUCE
2026-09-15 11:51:40,898 INFO pig.ExecTypeProvider: Picked MAPREDUCE as the ExecType
2026-09-15 11:51:41,281 [main] INFO org.apache.pig.Main - Apache Pig version 0.17.0 (r1797386) compiled Jun 02 2017, 15:41:58
2026-09-15 11:51:41,281 [main] INFO org.apache.pig.Main - Logging error messages to: C:\hadoop\logs\pig_1789451501278.log
2026-09-15 11:51:41,911 [main] INFO org.apache.pig.impl.util.Utils - Default bootup file C:\Users\moham/.pigbootup not found
2026-09-15 11:51:41,965 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapred.job.tracker is deprecated. Instead, use mapreduce.jobtracker.address
2026-09-15 11:51:41,965 [main] INFO org.apache.pig.backend.hadoop.executionengine.HExecutionEngine - Connecting to hadoop file system at: hdfs://localhost:9000
2026-09-15 11:51:42,466 [main] INFO org.apache.pig.PigServer - Pig Script ID for the session: PIG-sajid_ecommerce_analytics.pig-c21013c0-29d5-4159-a825-0676c6ed6
2026-09-15 11:51:42,466 [main] WARN org.apache.pig.PigServer - ATS is disabled since yarn.timeline-service.enabled set to false
2026-09-15 11:51:43,611 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapred.textoutputformat.separator is deprecated. Instead, use mapreduce.out
extoutputformat.separator
2026-09-15 11:51:43,646 [main] INFO org.apache.pig.tools.pigstats.ScriptState - Pig features used in the script: GROUP_BY,FILTER
2026-09-15 11:51:43,680 [main] INFO org.apache.pig.data.SchemaTupleBackend - Key [pig.schematuple] was not set... will not generate code.
2026-09-15 11:51:43,748 [main] INFO org.apache.pig.newplan.logical.optimizer.LogicalPlanOptimizer - {RULES_ENABLED=[AddForEach, ColumnMapKeyPrune, ConstantCalcula
GroupByConstParallelSetter, LimitOptimizer, LoadTypeCastInserter, MergeFilter, MergeForEach, NestedLimitOptimizer, PartitionFilterOptimizer, PredicatePushdownOptim
PushDownForEachFlatten, PushUpFilter, SplitFilter, StreamTypeCastInserter]}
2026-09-15 11:51:43,882 [main] INFO org.apache.pig.impl.util.SpillableMemoryManager - Selected heap (PS Old Gen) of size 699400192 to monitor. collectionUsageThre
= 489580128, usageThreshold = 489580128
2026-09-15 11:51:44,068 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MRCompiler - File concatenation threshold: 100 optimistic? false
2026-09-15 11:51:44,106 [main] INFO org.apache.pig.backend.hadoop.executionengine.util.CombinerOptimizerUtil - Choosing to move algebraic foreach to combiner
2026-09-15 11:51:44,116 [main] INFO org.apache.pig.backend.hadoop.executionengine.util.CombinerOptimizerUtil - Choosing to move algebraic foreach to combiner
2026-09-15 11:51:44,116 [main] INFO org.apache.pig.backend.hadoop.executionengine.util.CombinerOptimizerUtil - Choosing to move algebraic foreach to combiner
2026-09-15 11:51:44,122 [main] INFO org.apache.pig.backend.hadoop.executionengine.util.CombinerOptimizerUtil - Choosing to move algebraic foreach to combiner
2026-09-15 11:51:44,122 [main] INFO org.apache.pig.backend.hadoop.executionengine.util.CombinerOptimizerUtil - Choosing to move algebraic foreach to combiner
2026-09-15 11:51:44,147 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MultiQueryOptimizer - MR plan size before optimization: 6
2026-09-15 11:51:44,147 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MultiQueryOptimizer - Merged MR job 120 into MR job 117
2026-09-15 11:51:44,147 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MultiQueryOptimizer - Merged MR job 122 into MR job 117

```

Figure 9a — Screenshot: Pig session startup, ExecType selection (MAPREDUCE), and query-plan optimization (multi-query merge)

```

2026-09-15 11:51:45,886 [JobControl] INFO org.apache.pig.backend.hadoop.executionengine.util.MapRedUtil - Total input paths to process : 1
2026-09-15 11:51:45,930 [JobControl] INFO org.apache.pig.backend.hadoop.executionengine.util.MapRedUtil - Total input paths (combined) to process : 1
2026-09-15 11:51:46,067 [JobControl] INFO org.apache.hadoop.mapreduce.JobSubmitter - number of splits:1
2026-09-15 11:51:46,144 [JobControl] INFO org.apache.hadoop.conf.Configuration.deprecation - yarn.resourcemanager.system-metrics-publisher.enabled is deprecated, use yarn.system-metrics-publisher.enabled
2026-09-15 11:51:46,250 [JobControl] INFO org.apache.hadoop.mapreduce.JobSubmitter - Submitting tokens for job: job_1789449605750_0004
2026-09-15 11:51:46,253 [JobControl] INFO org.apache.hadoop.mapreduce.JobSubmitter - Executing with tokens: []
2026-09-15 11:51:46,394 [JobControl] INFO org.apache.hadoop.mapred.YARNRunner - Job jar is not present. Not adding any jar to the list of resources.
2026-09-15 11:51:46,458 [JobControl] INFO org.apache.hadoop.conf.Configuration - resource-types.xml not found
2026-09-15 11:51:46,458 [JobControl] INFO org.apache.hadoop.yarn.util.resource.ResourceUtils - Unable to find 'resource-types.xml'.
2026-09-15 11:51:46,538 [JobControl] INFO org.apache.hadoop.yarn.client.api.impl.YarnClientImpl - Submitted application application_1789449605750_0004
2026-09-15 11:51:46,623 [JobControl] INFO org.apache.hadoop.mapreduce.Job - The url to track the job: http://Sajid:8088/proxy/application_1789449605750_0004/
2026-09-15 11:51:46,623 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - HadoopJobId: job_1789449605750_0004
2026-09-15 11:51:46,623 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - Processing aliases cat_group1,cat_group2,cat_group3,category_avg_price,category_quantity,category_revenue,city_avg_revenue,city_group1,city_group2,city_group3,city_avg_revenue,city_group1[31,14],payment_revenue[39,18],pay_group[38,12] C: category_avg_price[11,21],cat_group1[10,13],category_quantity[18,20],cat_group2[17,13],category_revenue[25,19],cat_group3[24,13],city_avg_revenue[32,19],city_group1[31,14],payment_revenue[39,18],pay_group[38,12] R: category_avg_price[11,21],category_quantity[18,20],category_revenue[25,19],city_avg_revenue[32,19],payment_revenue[39,18]
2026-09-15 11:51:46,636 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - 0% complete
2026-09-15 11:51:46,636 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - Running jobs are [job_1789449605750_0004]
2026-09-15 11:52:19,203 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - 24% complete
2026-09-15 11:52:19,203 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - Running jobs are [job_1789449605750_0004]
2026-09-15 11:52:25,789 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - 33% complete
2026-09-15 11:52:25,789 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - Running jobs are [job_1789449605750_0004]
2026-09-15 11:52:26,805 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - 50% complete
2026-09-15 11:52:26,805 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - Running jobs are [job_1789449605750_0004]
2026-09-15 11:52:35,935 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - Running jobs are [job_1789449605750_0004]
2026-09-15 11:52:36,967 [main] INFO org.apache.hadoop.yarn.client.RMPProxy - Connecting to ResourceManager at /0.0.0.0:8032
2026-09-15 11:52:36,978 [main] INFO org.apache.hadoop.mapred.ClientServiceDelegate - Application state is completed. FinalApplicationStatus=SUCCEEDED. Redirect to job history server
2026-09-15 11:52:39,999 [main] INFO org.apache.hadoop.ipc.Client - Retrying connect to server: 0.0.0.0/0.0.0.0:10020. Already tried 0 time(s); retry policy is org.apache.hadoop.ipc.DefaultRetryPolicy$MaximumCountWithFixedSleep(maxRetries=10, sleepTime=1000 MILLISECONDS)

```

Figure 9b — Screenshot: job_1789449605750_0004 submitted and progressing through 0% → 24% → 33% → 50% → (100%) complete

Note: After the underlying MapReduce job reached completion (“Application state is completed”), Pig attempted to fetch final job statistics from the JobHistoryServer for its end-of-run summary report. Because the JobHistoryServer was not running in this environment, that specific connection was refused and Pig retried it repeatedly (visible as repeated “Retrying connect to server: 0.0.0.0/0.0.0.0:10020” messages), eventually prompting “Terminate batch job (Y/N)?”, which was answered Y. This retry loop is a cosmetic side-effect of the missing JobHistoryServer and does not affect job correctness — the underlying job had already completed successfully and written all five output directories, as confirmed immediately afterward by `hdfs dfs -ls` and by every `hdfs dfs -cat` shown below.

```

2026-09-15 11:57:37,818 [main] INFO org.apache.hadoop.mapred.ClientServiceDelegate - Application state is completed. Redirect to job history server
2026-09-15 11:57:40,880 [main] INFO org.apache.hadoop.ipc.Client - Retrying connect to server: 0.0.0.0/0.0.0.0:10020. Already tried 0 time(s); retry policy is org.apache.hadoop.ipc.DefaultRetryPolicy$MaximumCountWithFixedSleep(maxRetries=10, sleepTime=1000 MILLISECONDS)
2026-09-15 11:57:43,910 [main] INFO org.apache.hadoop.ipc.Client - Retrying connect to server: 0.0.0.0/0.0.0.0:10020. Already tried 0 time(s); retry policy is org.apache.hadoop.ipc.DefaultRetryPolicy$MaximumCountWithFixedSleep(maxRetries=10, sleepTime=1000 MILLISECONDS)
2026-09-15 11:57:46,949 [main] INFO org.apache.hadoop.ipc.Client - Retrying connect to server: 0.0.0.0/0.0.0.0:10020. Already tried 0 time(s); retry policy is org.apache.hadoop.ipc.DefaultRetryPolicy$MaximumCountWithFixedSleep(maxRetries=10, sleepTime=1000 MILLISECONDS)
2026-09-15 11:57:49,983 [main] INFO org.apache.hadoop.ipc.Client - Retrying connect to server: 0.0.0.0/0.0.0.0:10020. Already tried 0 time(s); retry policy is org.apache.hadoop.ipc.DefaultRetryPolicy$MaximumCountWithFixedSleep(maxRetries=10, sleepTime=1000 MILLISECONDS)
2026-09-15 11:57:53,016 [main] INFO org.apache.hadoop.ipc.Client - Retrying connect to server: 0.0.0.0/0.0.0.0:10020. Already tried 0 time(s); retry policy is org.apache.hadoop.ipc.DefaultRetryPolicy$MaximumCountWithFixedSleep(maxRetries=10, sleepTime=1000 MILLISECONDS)
Terminate batch job (Y/N)? Y

C:\big_data_project\sales_data\e-commerce_hadoop_project>hdfs dfs -ls /ecommerce/pig_output
Found 5 items
drwxr-xr-x - moham supergroup 0 2026-09-15 11:52 /ecommerce/pig_output/category_avg_price
drwxr-xr-x - moham supergroup 0 2026-09-15 11:52 /ecommerce/pig_output/category_quantity
drwxr-xr-x - moham supergroup 0 2026-09-15 11:52 /ecommerce/pig_output/category_revenue
drwxr-xr-x - moham supergroup 0 2026-09-15 11:52 /ecommerce/pig_output/city_avg_revenue
drwxr-xr-x - moham supergroup 0 2026-09-15 11:52 /ecommerce/pig_output/payment_revenue

```

Figure 9c — Screenshot: batch job terminated (Y) after the harmless history-server retries, followed by `hdfs dfs -ls /ecommerce/pig_output` confirming all 5 output directories exist

/ecommerce/pig_output

Go!

Show25entries

Search:

☐	Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name	
☐	drwxr-xr-x	moham	supergroup	0 B	Sep 15 11:52	0	0 B	category_avg_price	
☐	drwxr-xr-x	moham	supergroup	0 B	Sep 15 11:52	0	0 B	category_quantity	
☐	drwxr-xr-x	moham	supergroup	0 B	Sep 15 11:52	0	0 B	category_revenue	
☐	drwxr-xr-x	moham	supergroup	0 B	Sep 15 11:52	0	0 B	city_avg_revenue	
☐	drwxr-xr-x	moham	supergroup	0 B	Sep 15 11:52	0	0 B	payment_revenue	

Showing 1 to 5 of 5 entries

Previous

1

Next

Figure 9d — HDFS NameNode UI: /ecommerce/pig_output listing all 5 output directories, 15 Sep 11:52

9.1 Category Average Unit Price

Objective: Find the average unit price of products within each category. This is the Pig counterpart to the MapReduce CategoryAvgPriceMR job in Section 8.3, implemented independently for cross-checking.

Script

```
cat_group1 = GROUP clean_sales BY category;
category_avg_price = FOREACH cat_group1 GENERATE group AS category, AVG(clean_sales.unit_price) AS avg_price;
STORE category_avg_price INTO '/ecommerce/pig_output/category_avg_price' USING PigStorage(',');
```

Output

```
hdfs dfs -cat /ecommerce/pig_output/category_avg_price/part-r-00000
```

```
C:\big_data_project\sales_data\ecommerce_hadoop_project\pig>hdfs dfs -ls /ecommerce/pig_output
Found 5 items
drwxr-xr-x - moham supergroup      0 2026-09-15 11:52 /ecommerce/pig_output/category_avg_price
drwxr-xr-x - moham supergroup      0 2026-09-15 11:52 /ecommerce/pig_output/category_quantity
drwxr-xr-x - moham supergroup      0 2026-09-15 11:52 /ecommerce/pig_output/category_revenue
drwxr-xr-x - moham supergroup      0 2026-09-15 11:52 /ecommerce/pig_output/city_avg_revenue
drwxr-xr-x - moham supergroup      0 2026-09-15 11:52 /ecommerce/pig_output/payment_revenue

C:\big_data_project\sales_data\ecommerce_hadoop_project\pig>hdfs dfs -cat /ecommerce/pig_output/category_avg_price/part-r-00000
Toys,1002.0927884583291
Books,1010.4033408931801
Beauty,1002.1257454448274
Sports,1003.2304135044894
Grocery,1003.7117876763091
Clothing,1005.3702719797545
Furniture,1002.9672177802995
Automotive,1006.4333336677147
Electronics,1007.5722334889463
Home Appliances,1006.7754330787589
```

Figure 9.1 — Average unit price per category, plus the start of the category_quantity listing

Result Description

The result contains 10 rows, one average unit price per category, all clustered tightly in a narrow band:

- Lowest average unit price: Toys, at BDT 1,002.09.
- Highest average unit price: Books, at BDT 1,010.40.
- These figures match the independently-computed MapReduce result in Section 8.3 exactly to two decimal places for all ten categories — confirmed directly in Section 10.

9.2 Category Total Quantity

Objective: Find the total quantity of items sold within each category.

Script

```
cat_group2 = GROUP clean_sales BY category;  
category_quantity = FOREACH cat_group2 GENERATE group AS category, SUM(clean_sales.quantity) AS  
total_quantity;  
STORE category_quantity INTO '/ecommerce/pig_output/category_quantity' USING PigStorage(',');
```

Output

```
hdfs dfs -cat /ecommerce/pig_output/category_quantity/part-r-00000
```

```
C:\big_data_project\sales_data\ecommerce_hadoop_project\pig>hdfs dfs -cat /ecommerce/pig_output/category_quantit  
Toys,298854  
Books,299315  
Beauty,299577  
Sports,301912  
Grocery,302956  
Clothing,299924  
Furniture,297836  
Automotive,300353  
Electronics,298550  
Home Appliances,299811
```

Figure 9.2 — Total quantity sold per category

Result Description

- Highest total quantity: Grocery, with 302,956 units sold.
- Lowest total quantity: Furniture, with 297,836 units sold.
- All ten categories fall within a narrow band of roughly 297,800–303,000 units, a spread of under 2%.

9.3 Category Total Revenue

Objective: Find the total revenue generated within each category.

Script

```
cat_group3 = GROUP clean_sales BY category;  
category_revenue = FOREACH cat_group3 GENERATE group AS category, SUM(clean_sales.revenue) AS  
total_revenue;  
STORE category_revenue INTO '/ecommerce/pig_output/category_revenue' USING PigStorage(',');
```

Output

```
hdfs dfs -cat /ecommerce/pig_output/category_revenue/part-r-00000
```

```
C:\big_data_project\sales_data\ecommerce_hadoop_project\pig>hdfs dfs -cat /ecommerce/pig_output/category_revenue/
Toys,2.9945733836000097E8
Books,3.0239574375000143E8
Beauty,2.998436445300002E8
Sports,3.0291726385999924E8
Grocery,3.037467996799995E8
Clothing,3.016383198199985E8
Furniture,2.983833019899999E8
Automotive,3.0250336941000074E8
Electronics,3.0065963898000044E8
Home Appliances,3.014970277500006E8
```

Figure 9.3 — Total revenue per category (scientific notation, e.g. 3.0375E8 = BDT 303,746,799.68)

Result Description

- Highest total revenue: Grocery, with BDT 303,746,799.68 — consistent with Grocery also leading in total quantity (Section 9.2).
- Lowest total revenue: Furniture, with BDT 298,383,301.99 — also the lowest in total quantity.
- Revenue ranking closely tracks the quantity ranking, since average unit price varies by less than 1% between categories (Section 9.1).

9.4 City Average Revenue per Order

Objective: Find the average order revenue for each city.

Script

```
city_group1 = GROUP clean_sales BY city;
city_avg_revenue = FOREACH city_group1 GENERATE group AS city, AVG(clean_sales.revenue) AS
avg_revenue;
STORE city_avg_revenue INTO '/ecommerce/pig_output/city_avg_revenue' USING PigStorage(',');
```

Output

```
hdfs dfs -cat /ecommerce/pig_output/city_avg_revenue/part-r-00000
```

```
C:\big_data_project\sales_data\ecommerce_hadoop_project\pig>hdfs dfs -cat /ecommerce/pig_output/city_avg_revenue/part-r-00
Feni,5010.2111982308015
Bogra,5034.612822367576
Dhaka,5021.6445382691045
Pabna,4984.006879392669
Khulna,5030.7478895877075
Sylhet,5033.084573353573
Barisal,5036.265556288999
Comilla,5046.370551762796
Gazipur,5064.085763128198
Jessore,5038.1506014835595
Rangpur,4998.617977067723
Tangail,5017.971307191739
Dinajpur,5001.706214751259
Faridpur,5017.061837238799
Noakhali,4996.647504378887
Rajshahi,5044.665163398686
Chattogram,5019.609864179423
Coxs Bazar,5036.945783843244
Mymensingh,4990.085495113895
Mazumbari,5011.76782225126
```

Figure 9.4 — Average order revenue for the 20 cities (as captured on screen; window truncated the last entry)

Result Description

- Highest average order revenue: Gazipur, at BDT 5,064.09 per order.
- Lowest average order revenue: Pabna, at BDT 4,984.01 per order.
- The spread between the highest and lowest city average is only about 1.6%.
- Note that the city with the highest average revenue per order (Gazipur) is not the same as the city with the highest total revenue (Comilla, Section 8.1) — Comilla's higher total is driven by a slightly larger number of orders rather than a higher value per order.

9.5 Revenue by Payment Method

Objective: Find the total revenue generated through each payment method.

Script

```
pay_group = GROUP clean_sales BY payment_method;  
payment_revenue = FOREACH pay_group GENERATE group AS payment_method, SUM(clean_sales.revenue) AS  
total_revenue;  
STORE payment_revenue INTO '/ecommerce/pig_output/payment_revenue' USING PigStorage(',');
```

Output

```
hdfs dfs -cat /ecommerce/pig_output/payment_revenue/part-r-00000
```

```
C:\big_data_project\sales_data\ecommerce_hadoop_project\pig>hdfs dfs -cat /ecommerce/pig_output/payment_reve  
Card,7.543319097900054E8  
Cash,7.541534764499967E8  
Bank Transfer,7.511488471400067E8  
Mobile Banking,7.534082147499907E8  
  
C:\big_data_project\sales_data\ecommerce_hadoop_project\pig>
```

Figure 9.5 — Total revenue per payment method (scientific notation, e.g. 7.5433E8 = BDT 754,331,909.79)

Result Description

- Highest total revenue: Card, with BDT 754,331,909.79.
- Second: Cash, with BDT 754,153,476.45.
- Third: Mobile Banking, with BDT 753,408,214.75.
- Lowest total revenue: Bank Transfer, with BDT 751,148,847.14.

Interestingly, the revenue ranking does not exactly match the transaction-count ranking from Section 8.2: Mobile Banking has the highest transaction count (150,248 orders) but only the third-highest revenue, while Card has the second-highest transaction count (149,879 orders) but the highest total revenue. This indicates that although order counts are almost uniform across payment methods, the average order value differs slightly enough between methods to change the revenue ranking — a detail only visible by combining the MapReduce transaction-count result with the Pig revenue-by-payment-method result.

9.6 Pig Output Directory Structure

```
/ecommerce/pig_output/  
├── category_avg_price  
├── category_quantity  
├── category_revenue  
├── city_avg_revenue  
└── payment_revenue
```

This organization makes it straightforward to inspect, compare, and re-use the output of each Pig operation.

10. Discussion

This report documents eight analyses performed on a 600,000-record e-commerce transaction dataset: three implemented directly in Java MapReduce and five implemented in a single Apache Pig script. Because `sales_data.csv` is a single denormalized file in which every order already carries its city, category, and payment-method information, no JOIN operation was required for any of the eight analyses — each one is a straightforward FILTER/GROUP/aggregate pattern.

HDFS Storage

HDFS was used to store `sales_data.csv` (56.14 MB, 600,000 records) with a replication factor of 3 and a block size of 128 MB, providing the distributed storage layer that both the MapReduce jobs and the Pig script read from. Before the final run, all prior output directories (`mr_output_city`, `mr_output_payment`, `pig_output`) were explicitly deleted with `hdfs dfs -rm -r`, guaranteeing that every result in this report reflects a single, fresh, continuous execution rather than a mixture of runs from different sessions.

MapReduce Analysis — Three Real, Verified Jobs

The three MapReduce jobs — `CitySalesMR`, `PaymentMethodMR`, and `CategoryAvgPriceMR` — were compiled, packaged, and executed in sequence on the same cluster session, producing job IDs `job_1789449605750_0001`, `_0002`, and `_0003` respectively. `CitySalesMR` packs two metrics (quantity and revenue) into a single Text value emitted by the Mapper, and the Reducer unpacks and sums both independently. `PaymentMethodMR` is a simpler word-count-style job that counts transactions per payment method using an `IntWritable` accumulator. `CategoryAvgPriceMR` packs a (price, count) pair per record so the Reducer can compute a true average (total price ÷ total count) rather than an average of per-record averages, which would be mathematically incorrect if categories had unequal record counts. All three jobs report Map input records = 600,001 (600,000 data rows plus the header row) and Map output records = 600,000 (the header row is filtered out in the Mapper before being emitted), confirming every real order in the dataset was processed.

Apache Pig Analysis — One Script, One Merged Job

The five Pig operations were all implemented in a single script (`sajid_ecommerce_analytics.pig`) using the same GROUP + FOREACH + aggregate function pattern: category average unit price, category total quantity, category total revenue, city average revenue per order, and total revenue by payment method. Run in MAPREDUCE execution mode, Pig's `MultiQueryOptimizer` automatically merged what would

otherwise have been up to six separate underlying MapReduce jobs into a single job (job_1789449605750_0004), visible directly in the captured “Merged MR job 120 into MR job 117” log lines in Section 9. This is a concrete illustration of one of Pig's practical advantages: the developer writes five independent-looking aggregation blocks, but Pig's optimizer is free to execute them as efficiently as possible under the hood — in this case, as one pass over the data instead of five.

Real Cross-Validation: MapReduce vs. Pig on the Same Metric

Category average unit price was deliberately computed twice — once in MapReduce (CategoryAvgPriceMR, Section 8.3) and once in Pig (category_avg_price, Section 9.1) — specifically so the two independent implementations could be checked against each other. The results match exactly to two decimal places for all ten categories:

Category	MapReduce Result (BDT)	Pig Result (BDT)
Books	1,010.40	1,010.40
Electronics	1,007.57	1,007.57
Home Appliances	1,006.78	1,006.78
Automotive	1,006.43	1,006.43
Clothing	1,005.37	1,005.37
Grocery	1,003.71	1,003.71
Sports	1,003.23	1,003.23
Furniture	1,002.97	1,002.97
Beauty	1,002.13	1,002.13
Toys	1,002.09	1,002.09

This agreement is strong direct evidence that both the hand-written Java MapReduce aggregation logic and the Pig AVG() built-in are computing the same correct result over the same 600,000-row input — a genuine correctness cross-check enabled by having two independently-coded paths to the same answer, rather than two analyses that happen not to overlap.

The JobHistoryServer Retry Messages

The captured Pig console log in Section 9 includes a long sequence of “Retrying connect to server: 0.0.0.0/0.0.0.0:10020” messages after job_1789449605750_0004 reached completion. This occurred because the JobHistoryServer daemon was not running in this environment (it is optional and was not started alongside NameNode, DataNode, ResourceManager, and NodeManager — see Figure 5.2a). Pig tries to contact the JobHistoryServer purely to fetch final statistics for its own end-of-run summary printout; this has no effect on the job itself, which had already reached “Application state is completed” and “FinalApplicationStatus=SUCCEEDED” before the retries began. This is confirmed conclusively by the `hdfs dfs -ls /ecommerce/pig_output` listing showing all five expected output directories immediately afterward, and by every `hdfs dfs -cat` command in Section 9 successfully returning correct, non-empty results.

11. Conclusion

This project demonstrated a complete, end-to-end Big Data pipeline for a 600,000-record e-commerce sales dataset, using eight distinct analyses: three in Java MapReduce and five in Apache Pig, all executed on a real Hadoop cluster and documented with captured execution logs rather than reconstructed or simulated output.

The three MapReduce jobs (city-wise sales, payment-method usage, and category average price) demonstrated how custom Mapper and Reducer classes can compute a range of aggregation patterns directly in Java — from simple counting, to a composite value-encoding technique for tracking two metrics per key, to a (sum, count) accumulator pattern for computing a mathematically correct average.

The five Pig operations (category average price, category quantity, category revenue, city average revenue, and payment-method revenue) demonstrated how the same class of business question can be answered with significantly less code using Pig's GROUP and FOREACH constructs, all executed together as a single, optimizer-merged MapReduce job.

Deliberately computing category average unit price by both methods provided a genuine, real cross-validation: the MapReduce and Pig results matched exactly to two decimal places across all ten categories, giving strong confidence that both implementations are correct. Across all eight analyses, the results also showed that this dataset's orders are distributed close to uniformly across cities, categories, and payment methods (spreads of roughly 1–3% between the highest and lowest group in every analysis), with the one notable exception that the payment method generating the most revenue (Card) is not the same as the one used in the most transactions (Mobile Banking) — a finding only visible by combining the MapReduce transaction-count result with the Pig revenue-by-payment-method result.

Overall, this report provided a complete, verifiable demonstration of HDFS storage combined with both low-level (MapReduce) and high-level (Pig) distributed processing on a real transactional dataset, covering city-level, category-level, and payment-method-level aggregation, with every job, console log, counter, and output value traceable back to an actual execution on the cluster.